

Posto: Probabilistic Safety Monitoring

User Guide

1 What Posto Does

Posto lets you check if a dynamical system remains safe under uncertainty by:

1. Simulating trajectories

- **Equation mode:** next state from equations in a JSON file.
- **ANN mode:** next state from a trained neural network (.h5).

2. Generating uncertain logs

- From a single simulated trajectory.
- Each logged value is replaced by an interval $[lo, hi]$ of fixed half-width `--dtlog`.
- Only a random subset of time steps (probability `--prob%`) are logged, plus always $t = 0$.

3. Checking safety

- Safety constraints are read from a JSON file (`safety_constraints`).
- Both log samples and simulated trajectories consistent with the log are checked.
- Output is SAFE / UNSAFE plus plots.

2 Installation & Environment

2.1 Python Dependencies

Install the libraries actually used by this codebase:

```
pip install numpy matplotlib docopt tensorflow
```

You can also use keras directly instead of tensorflow; the code tries:

```
from tensorflow.keras.models import load_model
# falls back to:
from keras.models import load_model
```

2.2 Environment Variable: POSTO_ROOT_DIR

Many modules expect the project root via:

```
PROJECT_ROOT = os.environ['POSTO_ROOT_DIR']
sys.path.append(PROJECT_ROOT)
```

Set it once:

```
export POSTO_ROOT_DIR=/absolute/path/to/Posto
```

Add this export to `~/.bashrc` to make it permanent.

3 Repository Layout

```
Posto/  
  posto.py  
  System.py  
  Parameters.py  
  artEval.py  
  artEvalNN.py  
  art/  
    figA2a  
      img  
    figA2b  
      genTimePlot.py  
    figA2c  
      confidence.py  
    figA3a  
      img  
    figA3b  
      img  
    figA3c  
      img  
    figA3d  
      img  
    figA4a  
      img  
    figA4b  
      img  
    figA4c  
      img  
    figA4d  
      img  
    figB5  
      img  
    figB6a  
      img  
    figB6b  
      img  
    figB6c  
      img  
    figB6d  
      img  
  lib/  
    Equation.py  
    ANN.py
```

```
GenLog.py
TrajSafety.py
TrajValidity.py
JFBF.py
Visualize.py
models/
logs/
dev/
  ModelANN.py
  Model.py
```

Running the Posto Artifact (OVA)

This artifact is distributed as a pre-configured VirtualBox virtual machine to ensure full reproducibility of the experimental results reported in the paper.

1. Install Oracle VirtualBox (version 7.0 or later) from the official website: <https://www.virtualbox.org/wiki/Downloads> Please ensure that the **VirtualBox Extension Pack** corresponding to the same version is also installed.
2. Download the Posto zip from the following link and unzip the contents:
<https://alabama.box.com/s/6gn0u0anx0wm8tavhff21qy7clrjgs5b>
3. Open **VirtualBox Manager**
4. Select **File** → **Import Appliance**
5. Choose the downloaded .ova file from the unzipped contents
6. Click **Next**, then **Import**
7. Start the imported virtual machine

No additional configuration or installation is required.

Posto Location inside the Virtual Machine

```
~/Desktop/Posto
cd ~/Desktop/Posto
```

4 Model JSON Format

Each model JSON must include:

- `state_vars`

- constants (optional)
- ranges (optional)
- equations (required)
- safety_constraints

Example

```
{
  "state_vars": ["x", "y"],
  "constants": {"dt": 0.01},
  "ranges": {"ep": [-0.002, 0.002]},
  "equations": {
    "x'": "x + dt * (-y - (1.5*(x*x) - 0.5*(x*x*x) - 0.5)) + ep",
    "y'": "y + dt * ((3*x) - y) + ep"
  },
  "safety_constraints": [
    { "state": "x", "op": "le", "const": -0.10 },
    { "state": "y", "op": "le", "const": -0.10 }
  ]
}
```

4.1 Safety Constraint Semantics

Unsafe whenever constraint evaluates to true:

- "ge": state $\geq c$
- "le": state $\leq c$
- "gt": state $> c$
- "lt": state $< c$

5 Command-Line Interface

Commands

```
postto.py behavior
postto.py generateLog
postto.py checkSafety
```

Global Arguments

```
--log=<logfile> --mode=<equation|ann> --model_path=<path> --states=<comma-list>
--constraints=<json>
```

Descriptions

behavior — Simulates trajectories and saves plots under `logdir/img/`.

generateLog — Creates a log file with interval uncertainty.

checkSafety — Evaluates trajectories using the Jeffreys Bayes Factor.

6 Custom / Dev Mode

Minimal Custom Model

```
# Model.py
import random

def my_getNextState(state):
    x, y = state
    x = x + 0.1 + random.uniform(-0.01, 0.01)
    y = y - 0.05 + random.uniform(-0.01, 0.01)
    return (x, y)
```

Injecting Custom Dynamics

```
sys = System(log_path="logs/custom.lg", mode=None, model_path=None)
sys.getNextState = my_getNextState
init_box = [[0.0,1.0], [0.0,2.0]]
sys.generateLog(init_box, T=100, prob=50, dtlog=0.01)
```

Three-State Example

```
def my_getNext(state):
    x, y, z = state
    # Custom dynamics here
    return (x_next, y_next, z_next)

sys = System(log_path="logs/custom.lg",
             states=['x','y','z'])

sys.getNextState = my_getNext
sys.behaviour(init_box, T=1000)
```

```
sys.generateLog(init_box, T=2000, prob=2, dtlog=0.02)  
sys.checkSafety()
```